INTRODUCTION TO NEURAL NETWORK EXECUTION AND TRAINING[1]

TABLE OF CONTENTS

1. INTRODUCTION 2
2. NEURAL NETWORK REFERENCE SYSTEM 2
3. NEURAL NETWORK EXECUTION: FORWARD PROPAGATION. 3
4. NEURAL NETWORK EXECUTION: ERROR COMPUTATION 6
5. COMPUTING DEPENDENCE OF ERRORS ON WEIGHTS 7
6. NEURAL NETWORK TRAINING PROCESS 9
7. DERIVATIVE CALCULUS METHODS 10
   (a) Chain Rule 10
   (b) Quotient Rule 11
   (c) Power Rule 11
   (d) Exponential Derivative 11
8. DEPENDENCE OF NEURAL NETWORK ERROR ON WEIGHT VALUES . 11
9. NEURAL NETWORK GRADIENT DESCENT VISUALIZATION 17
10. OBSERVING THE GRADIENT DESCENT 23
11. LIMITATIONS OF GRADIENT DESCENT 25
12. REFERENCES 27

[1] This handout was developed by Prof. William Kaiser at the University of California, Los Angeles and is here used with the author's permission.

1. INTRODUCTION

This lecture is directed to provide a complete introduction to the operation and training of a Neural Network. The objective is to enable students to learn the most fundamental principles at the very foundation of Neural Network systems.

This lecture includes a reference neural network that while simplified, contains the most important components required to visualize and fully understand the computation provided by a trained neural network as well as the complete process of neural network *training*. This requires only a familiarity with derivative calculus. This lecture will include description of the fundamental methods required.

The author is indebted to inspiration for instruction by Matt Mazur[1] of Emergent Mind.

2. NEURAL NETWORK REFERENCE SYSTEM

The complete description of Neural Network computation and training will be based on a network structure including two input neurons, two output neurons and one hidden layer with two neurons as shown in Figure 1, below.

The output neurons are O_1 and O_2, the Hidden Layer Neurons are H_1 and H_2, and the input Neurons are I_1 and I_2.

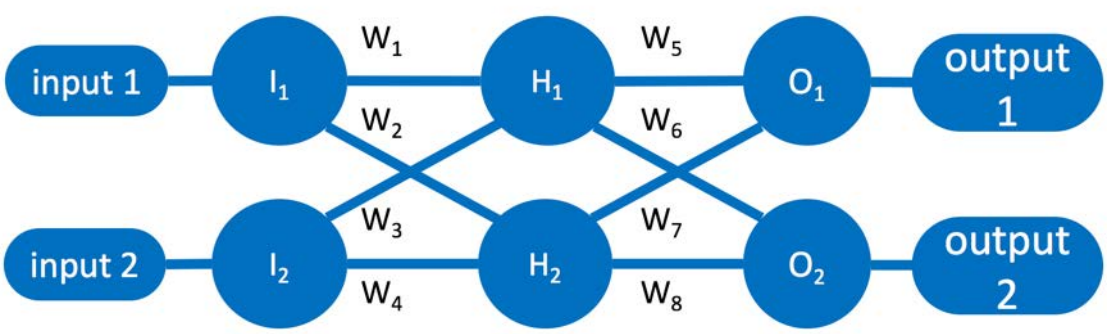

Figure 1. Neural Network Reference System

In this reference system, Weights are defined for each synapse connection. Also, Hidden Layer neurons include bias values, b_1, and Output Layer neurons include bias values, b_2. These will be described below. To simplify this operation and the following computation, these bias values will remain fixed. In general, of course, each neuron may include a unique bias value that is adjusted during training. This is not required here, however, to describe complete Neural Network computation and training.

3. NEURAL NETWORK EXECUTION: FORWARD PROPAGATION.

Neural Network computation of output values is based on the *Forward Propagation* algorithm. Here, the values produced by Input Layer neurons are multiplied by weight values and are presented to the input of Hidden Layer neurons as shown below.

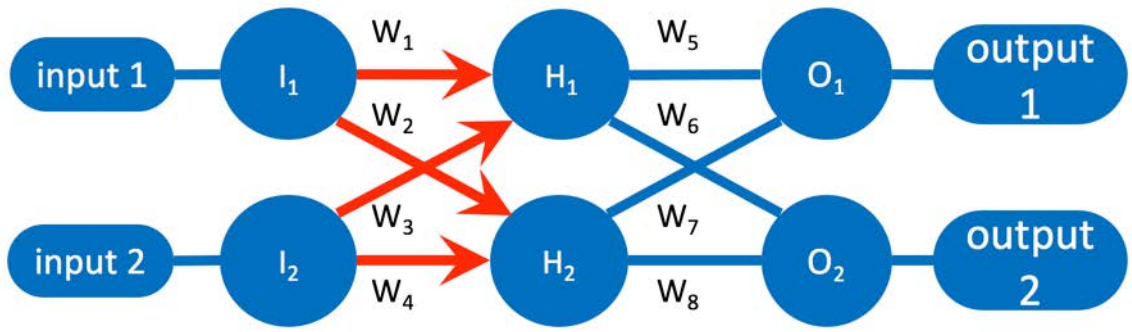

Figure 2. Neural Network Reference System with forward propagation of inputs to Hidden Layer neuron inputs.

Neuron structure is shown in Figure 3. Forward Propagation begins with the input values. These are multiplied by weights and provided to the inputs of the Hidden Layer neurons. These inputs are summed and then summed with Bias values. The result is supplied to the Activation Function. The Activation Function output appears as the Neuron Output.

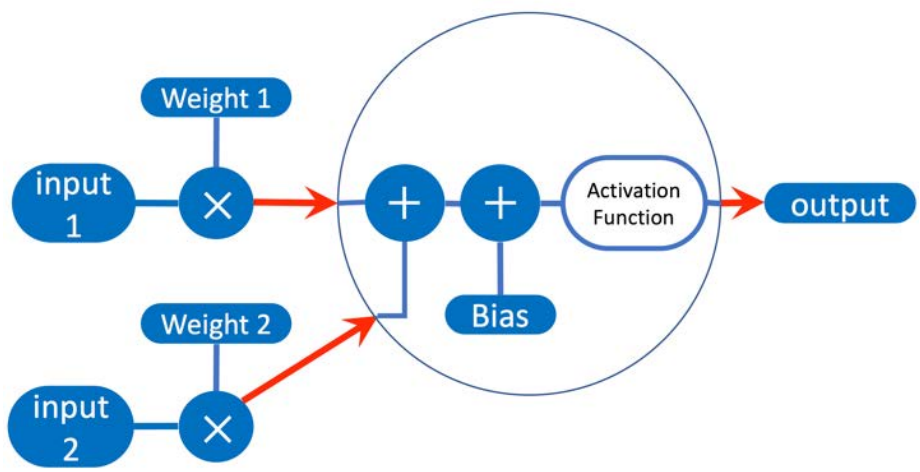

Figure 4. Neural Network individual neuron structure and operation. Input values are multiplied by weight values and summed. The Bias value is added to this summation and the result is supplied to the Activation Function. The Activation Function output appears as the Neuron Output.

The Sigmoid Activation Function for a single input and output is defined in Equation 2.

$$Output = \frac{1}{1 + e^{-Input}} \tag{1}$$

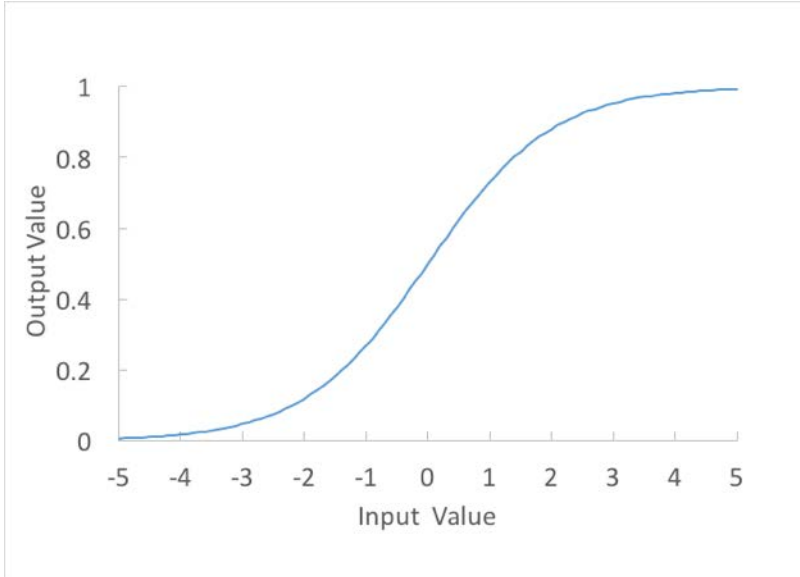

Figure 5. Neural Network individual neuron Sigmoid Activation Function.

Therefore, the Hidden Layer neuron of Figure 2 responds to the sum of its input values, $Input_j$, with an output defined as

$$Output = \frac{1}{1 + e^{-\left[\left(\sum_{j=1}^{N} Input_j\right) + b_1\right]}} \tag{2}$$

For this example, both Hidden Layer neurons will include bias values of $b_1$while Output Layer neurons will include bias values of b_2.

The computation of output values may proceed.

First, Hidden Layer neuron input values, $H_{1_{Input}}$ and $H_{2_{Input}}$ are computed for Input Layer Neuron values of I_1 and I_2.

$$H_{1_{Input}} = I_1 W_1 + I_2 W_3 + b_1 \tag{3}$$

$$H_{2_{Input}} = I_1 W_2 + I_2 W_4 + b_1 \tag{4}$$

Hidden Layer neuron output values are:

$$H_{1_{Output}} = \frac{1}{1 + e^{-H_{1_{input}}}} \tag{5}$$

$$H_{2_{Output}} = \frac{1}{1 + e^{-H_{2_{input}}}} \tag{6}$$

Continuing, Hidden Layer neuron outputs are applied to the inputs of the Output Layer Neurons as in Figure 6.

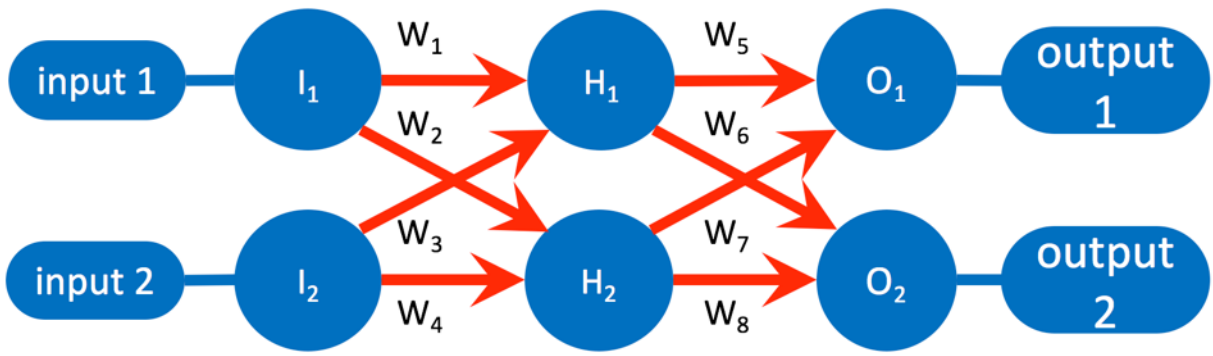

Figure 6. Neural Network Reference System with forward propagation of Hidden Layer outputs to the Output Neuron inputs.

Similarly, the input values at the Output Layer neurons are:

$$O_{1_{Input}} = H_{1_{output}} W_5 + H_{2_{output}} W_7 + b_2 \tag{7}$$

$$O_{2_{Input}} = H_{1_{output}} W_6 + H_{2_{output}} W_8 + b_2 \tag{8}$$

Finally, the Output Layer neuron outputs forming the Neural Network output of Figure 7 are:

$$O_{1_{Output}} = \frac{1}{1 + e^{-O_{1_{Input}}}} \tag{9}$$

$$O_{2_{Output}} = \frac{1}{1 + e^{-O_{2_{Input}}}} \tag{10}$$

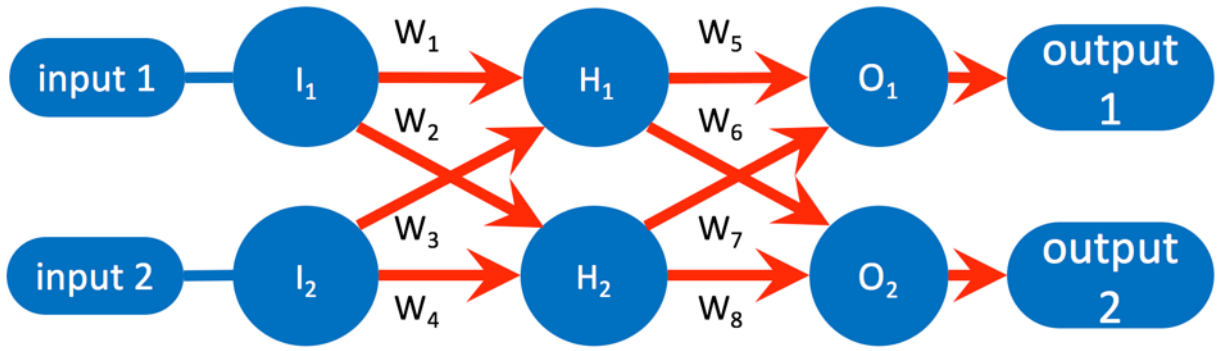

Figure 7. Neural Network Reference System with forward propagation of Output neuron final outputs.

4. NEURAL NETWORK EXECUTION: ERROR COMPUTATION

Engineering applications of Neural Networks include the development of models describing the behavior of complex systems corresponding to mechanical, electronic, environmental, biomedical, and other systems. The fundamental principles determining the system response may not be known. In addition, the response of a system may evolve in unpredictable ways as a result of the appearance of a defect, a change in an environment, or in the case of a biomedical systems, a disease condition.

However, experimental measurements will be available. Specifically, system output state, will be known by measurement for a set of known input values.

An ideal Neural Network would produce an output state for each set of input values with a response matching that of the complex system.

The known correct output states for a set of input values are referred to as Ground Truth.

In general, for a set of Weights and Bias values and any choice of Neural Network topology, output values may be computed for each set of input values.

Weight and Bias values, however, may be chosen or selected in advance and are not known to be those required for accurate computation of output states.

The Neural Network output error is then the difference between Ground Truth and neural network output corresponding to a set of inputs as shown in Figure 8.

An Error Cost Function for a set of M inputs and M known Ground Truth states depends on the selection of all Weight and Bias values. This is defined:

$$E(W,b) \equiv \frac{1}{2}\sum_{i=1}^{M}[Ground\ Truth(x_i) - Output(x_i)]^2 \tag{11}$$

The objective for Neural Network training is to find the set of Weight and Bias values that minimizes the Error Cost Function.

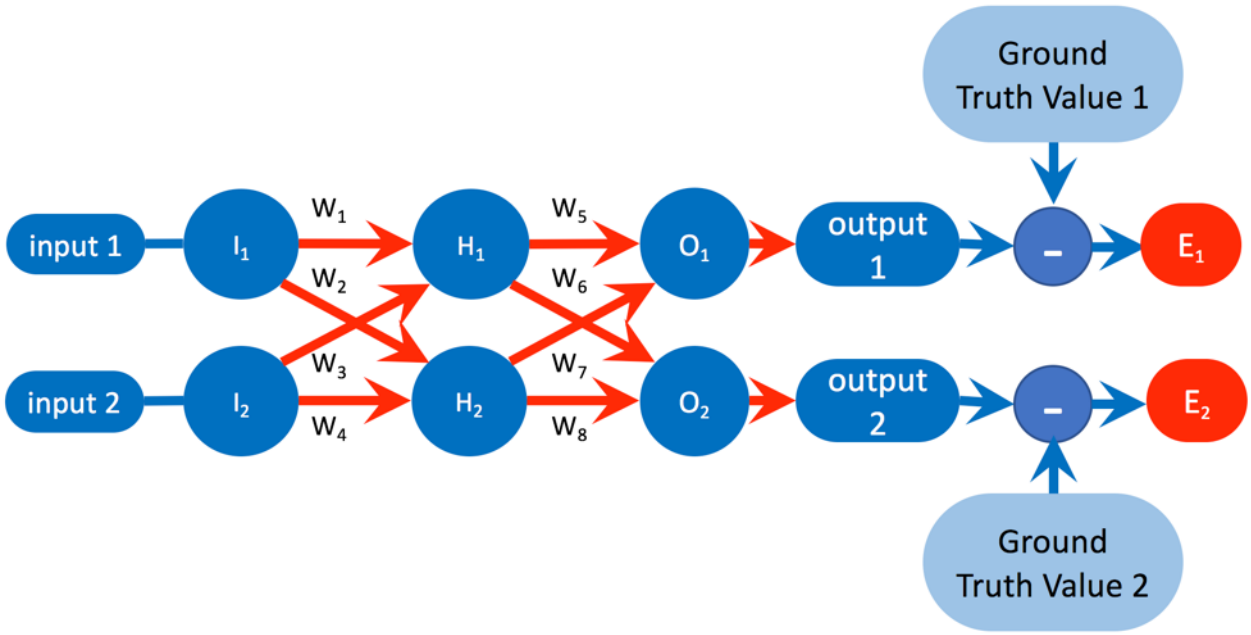

Figure 8. Neural Network Reference System with computation of errors that are the differences between Ground Truth (known correct values) and Outputs.

5. COMPUTING DEPENDENCE OF ERRORS ON WEIGHTS

The optimization of Weight and Bias values will begin with the determination of the dependence of output errors on Weight and Bias values. This process is Back Propagation.

A recursive Back Propagation process begins with computation of dependence of Errors on each set of Weights and Biases associated with Output Layer neurons and then for every layer between Output and Input as shown in Figures 9 through 11.

With known dependence of Errors on all Weights and Bias values, adjustment in these values may proceed with the objective of minimizing the Error Cost Function.

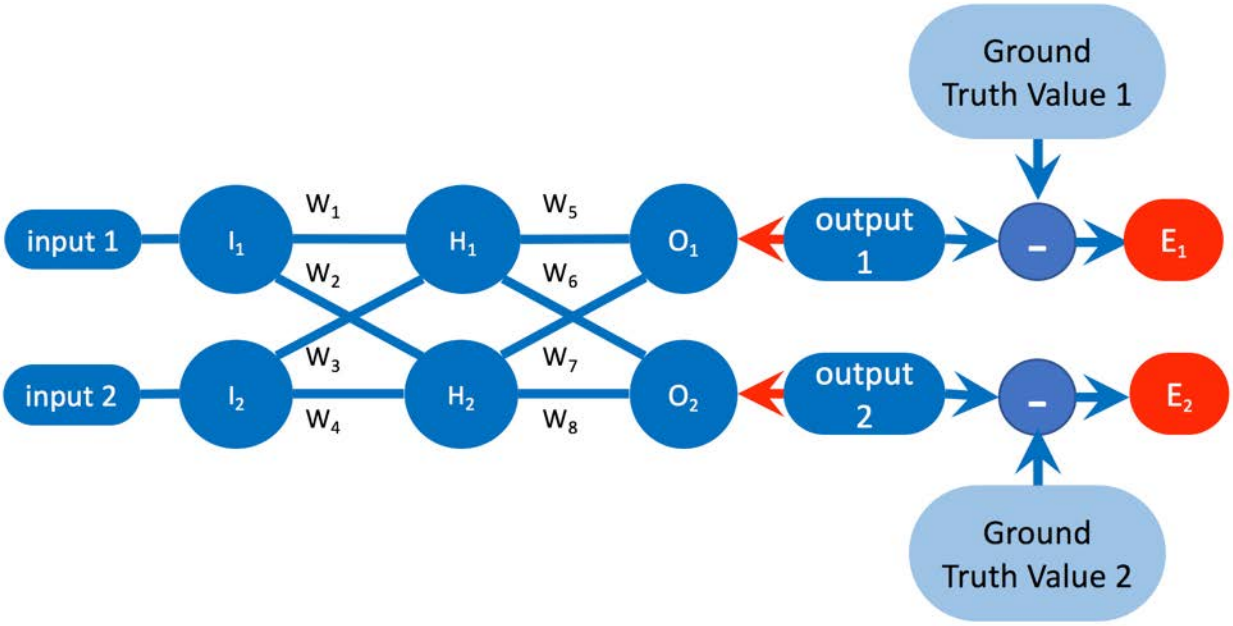

Figure 9. Neural Network Reference System with first step in back propagation of Output Error

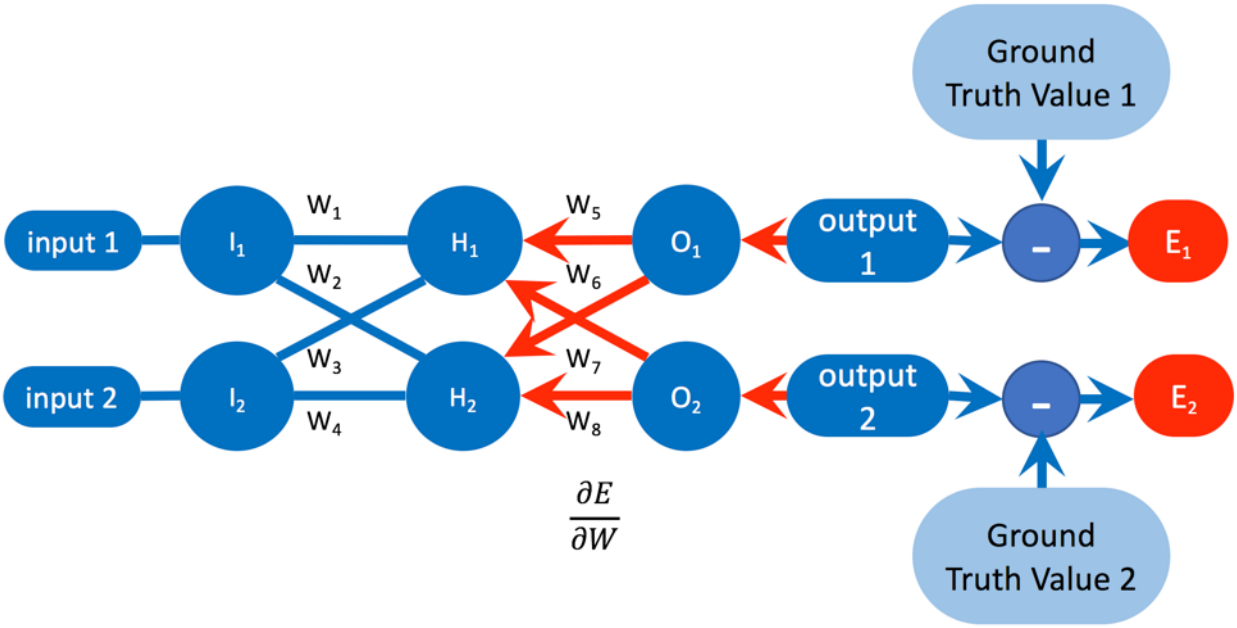

Figure 10. Neural Network Reference System with second step in back propagation of Output Error in determining dependence of Output Error on Weights W_5 through W_8.

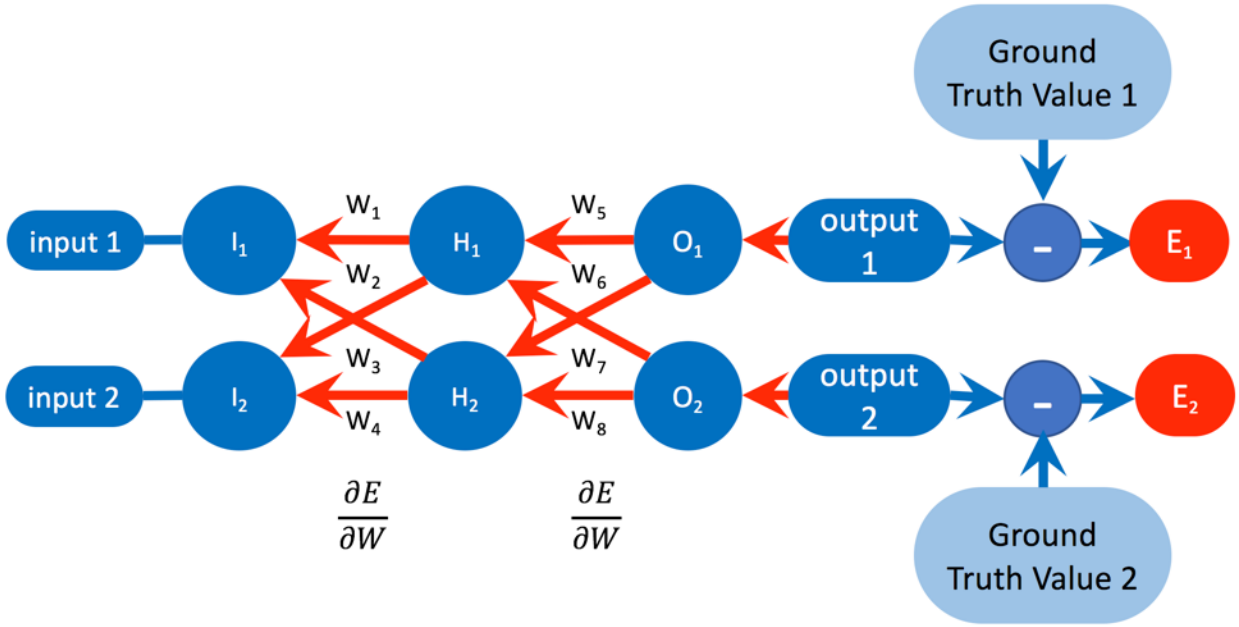

Figure 11. Neural Network Reference System with second step in back propagation of Output Error in determining dependence of Output Error on Weights W_1 through W_4.

6. NEURAL NETWORK TRAINING PROCESS

The most important attribute of Neural Network systems is the capability of *learning*. The Neural training process enables the Neural Network to classify input values to compute output values matching the known Ground Truth.

One of the most important Neural Network training methods depends on the Gradient Descent algorithm. Here, the partial derivative of Error on Weight is computed for each Weight. The sign and magnitude of the partial derivative indicates the sign and magnitude of the adjustment in Weight and Bias value that may reduce the Error Cost Function value.

The Gradient Descent training process begins with a selection of random values of Weights and Bias for each layer and each Neuron. Training results in a computation of Weight and Bias values that enable the Neural Network to classify inputs to match output Ground Truth.

In the Gradient Descent process, shown in Figure 12, a computation of the partial derivatives of Error on Weight are computed for each Weight. Then, the Weight values are modified by incremental amounts according to the scaled partial derivative values. The partial derivative values are scaled to ensure that the change in Weight values is incremental only. Each episode of computation of partial derivatives and incremental change in Weights is referred to as an epoch. After completion of each epoch, the new Error Cost Function is computed.

The Gradient Descent process continues in a series of epochs. The process continues until the Error Cost Function reaches a low value that is reduced no further by continued epoch execution. Alternatively, a designer may select another condition for termination of Gradient Descent based on the Error Cost Function reaching a low threshold value or a computation time being exceeded.

In the above, the change in Weight values occurring in each epoch depends on the scaling factor applied to the partial derivatives. This scaling factor is referred to as a Learning Rate.

A wide range of important Neural Network applications are served by the Gradient Descent process. However, applications appear where Gradient Descent methods are not successful in finding the true minimum Error Cost Function value. This will be described in the last section of this lecture.

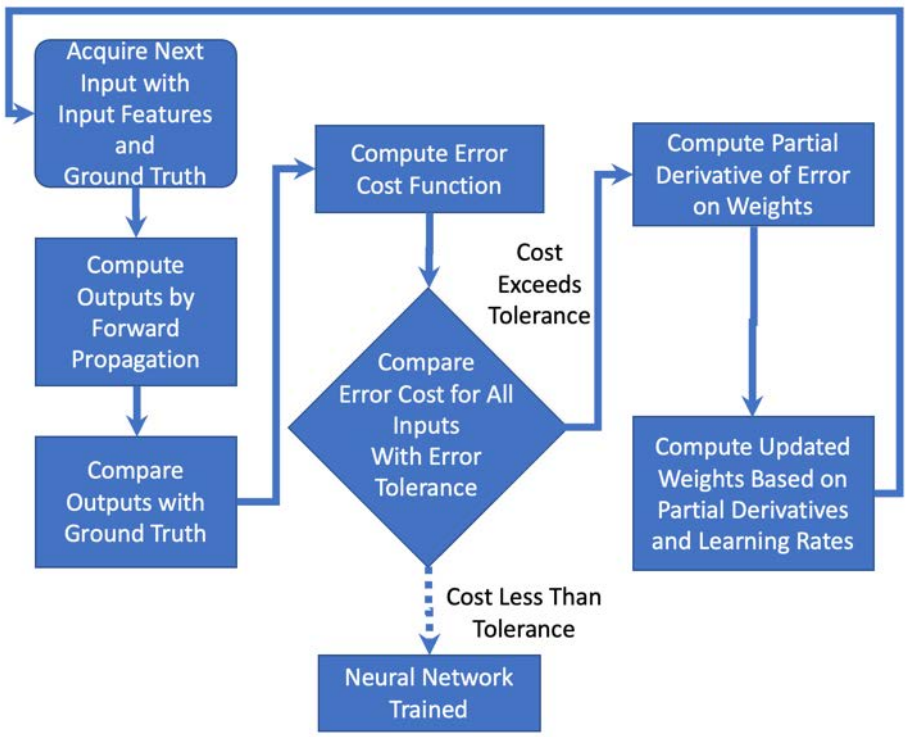

Figure 12. The Neural Network system Gradient Descent training sequence. This sequence starts with an initial set of Weights and Bias values that are randomly selected.

7. DERIVATIVE CALCULUS METHODS

Now, the computation of the partial derivatives of the Error on Weights is straightforward. Their computation is assisted by a set of familiar rules of differential calculus. These are summarized below.

(a) Chain Rule

$$h(x) = f(g(x)) \tag{12}$$

$$\left(\frac{\partial h(x)}{\partial x}\right) = \left(\frac{\partial f}{\partial g(x)}\right)\left(\frac{\partial g(x)}{\partial x}\right) \tag{13}$$

(b) Quotient Rule

$$h(x) = \frac{f(x)}{g(x)} \tag{14}$$

$$\left(\frac{\partial h(x)}{\partial x}\right) = \frac{\left(\frac{\partial f}{\partial x}\right) g(x) - f(x)\left(\frac{\partial g(x)}{\partial x}\right)}{g(x)^2} \tag{15}$$

(c) Power Rule

$$h(x) = f(x)^n \tag{16}$$

$$\left(\frac{\partial h(x)}{\partial x}\right) = nf(x)^{n-1}\left(\frac{\partial f}{\partial x}\right) \tag{17}$$

(d) Exponential Derivative

$$h(x) = e^{g(x)} \tag{18}$$

$$\left(\frac{\partial h(x)}{\partial x}\right) = e^{g(x)}\left(\frac{\partial g(x)}{\partial x}\right) \tag{19}$$

8. DEPENDENCE OF NEURAL NETWORK ERROR ON WEIGHT VALUES

First, it is necessary to compute the partial derivative of Error with respect to each Weight. Referring to Figures 9 through 11, These partial derivatives for the Weight values between the Hidden and Output layer neurons are:

$$\left(\frac{\partial E}{\partial W_5}\right); \left(\frac{\partial E}{\partial W_6}\right); \left(\frac{\partial E}{\partial W_7}\right); \left(\frac{\partial E}{\partial W_8}\right) \tag{20}$$

The derivative chain rule may be applied to compute each partial derivative in terms of partial derivatives that may be directly solved from the neural network topology, assignment of weights and description of activation functions.

$$\left(\frac{\partial E}{\partial W_5}\right) = \left(\frac{\partial E}{\partial O_{1_{Output}}}\right)\left(\frac{\partial O_{1_{Output}}}{\partial O_{1_{Input}}}\right)\left(\frac{\partial O_{1_{Input}}}{\partial W_5}\right) \tag{21}$$

$$\left(\frac{\partial E}{\partial W_6}\right) = \left(\frac{\partial E}{\partial O_{2_{Output}}}\right)\left(\frac{\partial O_{2_{Output}}}{\partial O_{2_{Input}}}\right)\left(\frac{\partial O_{2_{Input}}}{\partial W_6}\right) \tag{22}$$

$$\left(\frac{\partial E}{\partial W_7}\right) = \left(\frac{\partial E}{\partial O_{1_{Output}}}\right)\left(\frac{\partial O_{1_{Output}}}{\partial O_{1_{Input}}}\right)\left(\frac{\partial O_{1_{Input}}}{\partial W_7}\right) \tag{23}$$

$$\left(\frac{\partial E}{\partial W_8}\right) = \left(\frac{\partial E}{\partial O_{2_{Output}}}\right)\left(\frac{\partial O_{2_{Output}}}{\partial O_{2_{Input}}}\right)\left(\frac{\partial O_{2_{Input}}}{\partial W_8}\right) \tag{24}$$

Each of the partial derivatives included above, may be computed. First, Error, E, is defined in terms of both output errors,

$$E = E_1 + E_2 \tag{25}$$

Each output error is determined by the difference between ground truth, T_1 and T_2 and outputs, $O_{1_{Output}}$ and $O_{2_{Output}}$.

$$E_1 = \left(\frac{1}{2}\right)\left(T_1 - O_{1_{Output}}\right)^2 \tag{26}$$

$$E_2 = \left(\frac{1}{2}\right)\left(T_2 - O_{2_{Output}}\right)^2 \tag{27}$$

The partial derivatives of output error with respect to outputs are:

$$\left(\frac{\partial E}{\partial O_{1_{Output}}}\right) = \left(\frac{1}{2}\right)(2)\left(T_1 - O_{1_{Output}}\right)\left(\frac{\partial}{\partial O_{1_{Output}}}\right)\left(T_1 - O_{1_{Output}}\right) = -\left(T_1 - O_{1_{Output}}\right) \tag{28}$$

$$\left(\frac{\partial E}{\partial O_{1_{Output}}}\right) = -\left(T_1 - O_{1_{Output}}\right) \tag{29}$$

$$\left(\frac{\partial E}{\partial O_{2_{Output}}}\right) = -\left(T_2 - O_{2_{Output}}\right) \tag{30}$$

Next, the partial derivatives of Neuron output on Neuron input depends on the Activation Function. First, the output depends on input by the Sigmoid Activation Function define as:

$$O_{1_{Output}} = \frac{1}{1 + e^{-O_{1_{Input}}}} \tag{31}$$

The partial derivative is

$$\left(\frac{\partial O_{1_{Output}}}{\partial O_{1_{Input}}}\right) = \frac{\partial}{\partial O_{1_{Input}}}\left(\frac{1}{1 + e^{-O_{1_{Input}}}}\right) = -\frac{\frac{\partial}{\partial O_{1_{Input}}}\left(1 + e^{-O_{1_{Input}}}\right)}{\left(1 + e^{-O_{1_{Input}}}\right)^2} \tag{32}$$

$$\left(\frac{\partial O_{1_{Output}}}{\partial O_{1_{Input}}}\right) = -\frac{\frac{\partial}{\partial O_{1_{Input}}}\left(1 + e^{-O_{1_{Input}}}\right)}{\left(1 + e^{-O_{1_{Input}}}\right)^2} = -\frac{-e^{-O_{1_{Input}}}}{\left(1 + e^{-O_{1_{Input}}}\right)^2} = \frac{e^{-O_{1_{Input}}}}{\left(1 + e^{-O_{1_{Input}}}\right)^2} \tag{33}$$

Now, this result may be divided into two terms

$$\frac{e^{-O_{1_{Input}}}}{\left(1 + e^{-O_{1_{Input}}}\right)^2} = \frac{\left(1 + e^{-O_{1_{Input}}}\right) - 1}{\left(1 + e^{-O_{1_{Input}}}\right)^2} = \frac{1}{\left(1 + e^{-O_{1_{Input}}}\right)} - \left(\frac{1}{\left(1 + e^{-O_{1_{Input}}}\right)}\right)^2 \tag{34}$$

and this result can now be written directly as:

$$\frac{1}{\left(1+e^{-O_{1_{Input}}}\right)}-\left(\frac{1}{\left(1+e^{-O_{1_{Input}}}\right)}\right)^2=O_{1_{Output}}-\left(O_{1_{Output}}\right)^2=O_{1_{Output}}\left(1-O_{1_{Output}}\right) \quad (35)$$

Finally, each of the partial derivatives are:

$$\left(\frac{\partial O_{1_{Output}}}{\partial O_{1_{Input}}}\right)=\frac{\partial}{\partial O_{1_{Input}}}\left(\frac{1}{1+e^{-O_{1_{Input}}}}\right)=O_{1_{Output}}\left(1-O_{1_{Output}}\right) \quad (36)$$

$$\left(\frac{\partial O_{2_{Output}}}{\partial O_{2_{Input}}}\right)=\frac{\partial}{\partial O_{2_{Input}}}\left(\frac{1}{1+e^{-O_{2_{Input}}}}\right)=O_{2_{Output}}\left(1-O_{2_{Output}}\right) \quad (37)$$

$$\left(\frac{\partial H_{1_{Output}}}{\partial H_{1_{Input}}}\right)=\frac{\partial}{\partial H_{1_{Input}}}\left(\frac{1}{1+e^{-H_{1_{Input}}}}\right)=H_{1_{Output}}\left(1-H_{1_{Output}}\right) \quad (38)$$

$$\left(\frac{\partial H_{2_{Output}}}{\partial H_{2_{Input}}}\right)=\frac{\partial}{\partial H_{2_{Input}}}\left(\frac{1}{1+e^{-H_{2_{Input}}}}\right)=H_{2_{Output}}\left(1-H_{2_{Output}}\right) \quad (39)$$

The partial derivative of neuron inputs with respect to weights are:

$$\left(\frac{\partial O_{1_{Input}}}{\partial W_5}\right)=H_{1_{output}} \quad (40)$$

$$\left(\frac{\partial O_{1_{Input}}}{\partial W_7}\right)=H_{2_{output}} \quad (41)$$

$$\left(\frac{\partial O_{2_{Input}}}{\partial W_6}\right)=H_{1_{output}} \quad (42)$$

$$\left(\frac{\partial O_{2_{Input}}}{\partial W_8}\right)=H_{2_{output}} \quad (43)$$

Finally, the partial derivatives are:

$$\left(\frac{\partial E}{\partial W_5}\right)=\left[-\left(T_1-O_{1_{Output}}\right)\right]\left[O_{1_{Output}}\left(1-O_{1_{Output}}\right)\right]\left(H_{1_{output}}\right) \quad (44)$$

$$\left(\frac{\partial E}{\partial W_7}\right)=\left[-\left(T_1-O_{1_{Output}}\right)\right]\left[O_{1_{Output}}\left(1-O_{1_{Output}}\right)\right]\left(H_{2_{output}}\right) \quad (45)$$

$$\left(\frac{\partial E}{\partial W_6}\right)=\left[-\left(T_2-O_{2_{Output}}\right)\right]\left[O_{2_{Output}}\left(1-O_{2_{Output}}\right)\right]\left(H_{1_{output}}\right) \quad (46)$$

$$\left(\frac{\partial E}{\partial W_8}\right)=\left[-\left(T_2-O_{2_{Output}}\right)\right]\left[O_{2_{Output}}\left(1-O_{2_{Output}}\right)\right]\left(H_{2_{output}}\right) \quad (47)$$

The remaining step is to determine the partial derivatives of output Error on the Weights appearing at the input of the Hidden Layer Neurons. These are:

$$\left(\frac{\partial E}{\partial W_1}\right);\ \left(\frac{\partial E}{\partial W_2}\right);\ \left(\frac{\partial E}{\partial W_3}\right);\ \left(\frac{\partial E}{\partial W_4}\right) \tag{48}$$

Again, the derivative chain rule may be applied to compute each partial derivative in terms of partial derivatives that may be directly solved from the neural network topology, assignment of weights and description of activation functions. Referring to Figures 9 through 11, these partial derivatives may be computed:

$$\left(\frac{\partial E}{\partial W_1}\right) = \left(\frac{\partial E}{\partial H_{1_{Output}}}\right)\left(\frac{\partial H_{1_{Output}}}{\partial H_{1_{Input}}}\right)\left(\frac{\partial H_{1_{Input}}}{\partial W_1}\right) \tag{49}$$

$$\left(\frac{\partial E}{\partial W_3}\right) = \left(\frac{\partial E}{\partial H_{1_{Output}}}\right)\left(\frac{\partial H_{1_{Output}}}{\partial H_{1_{Input}}}\right)\left(\frac{\partial H_{1_{Input}}}{\partial W_3}\right) \tag{50}$$

$$\left(\frac{\partial E}{\partial W_2}\right) = \left(\frac{\partial E}{\partial H_{2_{Output}}}\right)\left(\frac{\partial H_{2_{Output}}}{\partial H_{2_{Input}}}\right)\left(\frac{\partial H_{2_{Input}}}{\partial W_2}\right) \tag{51}$$

$$\left(\frac{\partial E}{\partial W_4}\right) = \left(\frac{\partial E}{\partial H_{2_{Output}}}\right)\left(\frac{\partial H_{2_{Output}}}{\partial H_{2_{Input}}}\right)\left(\frac{\partial H_{2_{Input}}}{\partial W_4}\right) \tag{52}$$

The partial derivatives of error with respect to Hidden Layer Neuron outputs is computed for each of the error contributions

$$\frac{\partial E}{\partial H_{1_{Output}}} = \frac{\partial E_1}{\partial H_{1_{Output}}} + \frac{\partial E_2}{\partial H_{1_{Output}}} \tag{53}$$

$$\frac{\partial E}{\partial H_{2_{Output}}} = \frac{\partial E_1}{\partial H_{2_{Output}}} + \frac{\partial E_2}{\partial H_{2_{Output}}} \tag{54}$$

Then,

$$\frac{\partial E_1}{\partial H_{1_{Output}}} = \left(\frac{\partial E_1}{\partial O_{1_{Output}}}\right)\left(\frac{\partial O_{1_{Output}}}{\partial O_{1_{Input}}}\right)\left(\frac{\partial O_{1_{Input}}}{\partial H_{1_{Ouput}}}\right) \tag{55}$$

$$\frac{\partial E_2}{\partial H_{1_{Output}}} = \left(\frac{\partial E_2}{\partial O_{2_{Output}}}\right)\left(\frac{\partial O_{2_{Output}}}{\partial O_{2_{Input}}}\right)\left(\frac{\partial O_{2_{Input}}}{\partial H_{1_{Ouput}}}\right) \tag{56}$$

$$\frac{\partial E_1}{\partial H_{2_{Output}}} = \left(\frac{\partial E_1}{\partial O_{1_{Output}}}\right)\left(\frac{\partial O_{1_{Output}}}{\partial O_{1_{Input}}}\right)\left(\frac{\partial O_{1_{Input}}}{\partial H_{2_{Ouput}}}\right) \tag{57}$$

$$\frac{\partial E_2}{\partial H_{2_{Output}}} = \left(\frac{\partial E_2}{\partial O_{2_{Output}}}\right)\left(\frac{\partial O_{2_{Output}}}{\partial O_{2_{Input}}}\right)\left(\frac{\partial O_{2_{Input}}}{\partial H_{2_{Ouput}}}\right) \tag{58}$$

The partial derivatives of output error contributions on Output Layer Neuron outputs are

$$\left(\frac{\partial E_1}{\partial O_{1_{Output}}}\right) = \frac{\partial}{\partial O_{1_{Output}}}\left(\frac{1}{2}\right)\left(T_1 - O_{1_{Output}}\right)^2 = -\left(T_1 - O_{1_{Output}}\right) \tag{59}$$

Then, similarly for each derivative and also applying our previously computed result for the partial derivative of output Error on Output Layer Neuron output value,

$$\left(\frac{\partial E_1}{\partial O_{1_{Output}}}\right) = -\left(T_1 - O_{1_{Output}}\right) \tag{60}$$

$$\left(\frac{\partial E_2}{\partial O_{2_{Output}}}\right) = -\left(T_2 - O_{2_{Output}}\right) \tag{61}$$

$$\left(\frac{\partial H_{1_{Input}}}{\partial W_1}\right) = I_1 \tag{62}$$

$$\left(\frac{\partial H_{1_{Input}}}{\partial W_3}\right) = I_2 \tag{63}$$

$$\left(\frac{\partial H_{2_{Input}}}{\partial W_2}\right) = I_1 \tag{64}$$

$$\left(\frac{\partial H_{2_{Input}}}{\partial W_4}\right) = I_2 \tag{65}$$

Then, the partial derivatives of Hidden Layer Neuron outputs on Hidden Layer Neuron inputs are also determined by the Sigmoid Activation functions. These are computed in a way similar to that of the partial derivatives of Output Layer Neuron outputs on Output Layer Neuron inputs. Thus, these are:

$$\left(\frac{\partial H_{1_{Output}}}{\partial H_{1_{Input}}}\right) = H_{1_{Output}}\left(1 - H_{1_{Output}}\right) \tag{66}$$

$$\left(\frac{\partial H_{2_{Output}}}{\partial H_{2_{Input}}}\right) = H_{2_{Output}}\left(1 - H_{2_{Output}}\right) \tag{67}$$

Then, the partial derivatives of output Error on Hidden Layer neuron outputs are:

$$\frac{\partial E_1}{\partial H_{1_{Output}}} = \left[-\left(T_1 - O_{1_{Output}}\right)\right]\left[O_{1_{Output}}\left(1 - O_{1_{Output}}\right)\right][W_5] \tag{68}$$

$$\frac{\partial E_2}{\partial H_{1_{Output}}} = \left[-\left(T_2 - O_{2_{Output}}\right)\right]\left[O_{2_{Output}}\left(1 - O_{2_{Output}}\right)\right][W_6] \tag{69}$$

$$\frac{\partial E_1}{\partial H_{2_{Output}}} = \left[-\left(T_1 - O_{1_{Output}}\right)\right]\left[O_{1_{Output}}\left(1 - O_{1_{Output}}\right)\right][W_7] \tag{70}$$

$$\frac{\partial E_2}{\partial H_{2_{Output}}} = \left[-\left(T_2 - O_{2_{Output}}\right)\right]\left[O_{2_{Output}}\left(1 - O_{2_{Output}}\right)\right][W_8] \tag{71}$$

Then, recalling

$$\left(\frac{\partial E}{\partial H_{1_{Output}}}\right) = \frac{\partial E_1}{\partial H_{1_{Output}}} + \frac{\partial E_2}{\partial H_{1_{Output}}}$$

$$\left(\frac{\partial E}{\partial H_{2_{Output}}}\right) = \frac{\partial E_1}{\partial H_{2_{Output}}} + \frac{\partial E_2}{\partial H_{2_{Output}}}$$

With the above equations 68 through 72 and equations 54 and 55, all of the partial derivatives have been computed.

Finally, with the above results, the remaining four partial derivatives of Error on weights are:

$$\left(\frac{\partial E}{\partial W_1}\right) = \left(\frac{\partial E}{\partial H_{1_{Output}}}\right)\left[H_{1_{Output}}\left(1 - H_{1_{Output}}\right)\right][I_1] \tag{72}$$

$$\left(\frac{\partial E}{\partial W_3}\right) = \left(\frac{\partial E}{\partial H_{1_{Output}}}\right)\left[H_{1_{Output}}\left(1 - H_{1_{Output}}\right)\right][I_2] \tag{73}$$

$$\left(\frac{\partial E}{\partial W_2}\right) = \left(\frac{\partial E}{\partial H_{2_{Output}}}\right)\left[H_{2_{Output}}\left(1 - H_{2_{Output}}\right)\right][I_1] \tag{74}$$

$$\left(\frac{\partial E}{\partial W_4}\right) = \left(\frac{\partial E}{\partial H_{2_{Output}}}\right)\left[H_{2_{Output}}\left(1 - H_{2_{Output}}\right)\right][I_2] \tag{75}$$

9. NEURAL NETWORK GRADIENT DESCENT VISUALIZATION

The response of this Neural Network to applied inputs and its training can now be directly computed.

First, the Neural Network will be trained to recognize two inputs and two corresponding outputs. The Neural Network will be trained with the inputs and Ground Truth values of Table 1.

States	Inputs		Ground Truth	
	Input 1	Input 2	Output 1	Output 2
State 1	0.0	1.0	0.0	1.0
State 2	1.0	0.0	1.0	0.0

Table 1. Input Values and Ground Truth Output Values for two States. The Neural Network will be trained to classify these two inputs to produce outputs matching Ground Truth.

A critical step is the initial selection of Weight and Bias values. The first example of training will apply the randomly selected Weight values of Table 2.

Weight	W_1	W_2	W_3	W_4	W_5	W_6	W_7	W_8
Value	-1.5	-3.5	2.0	-1.0	2.0	-3.0	4.0	-1.25

Table 2. Initial Weight Values for the Neural Network at initiation of training.

As has been described, this Neural Network includes fixed bias values of b_1 and b_2 . For the purpose of this computation, the Bias values will be held constant at the value of 0.5 and only Weight value modification is required for training as described previously.

The Neural Network training process of Figure 12 starts with the Weight values of Table 2. The two input States are supplied and outputs are compared to the Ground Truth of Table 1. The Weight values and Output Values are computed at each Epoch and the results are plotted in Figure 13.

Figure 13 shows that Output 1 for State 1 is initially computed at a value of over 0.75, while Ground Truth value is 0.0. However, as training proceeds through each Epoch, the value of Output 1 for State 1 falls to zero. The value of Output 1 for State 2 is initially small. However, as Epochs occur, its value rises to the Ground Truth value of 1.0. The values of Weights 1 through 4 are also observed in Figure 13. The Back Propagation and Gradient Descent process adjusts the Weight values to enable accurate classification of inputs.

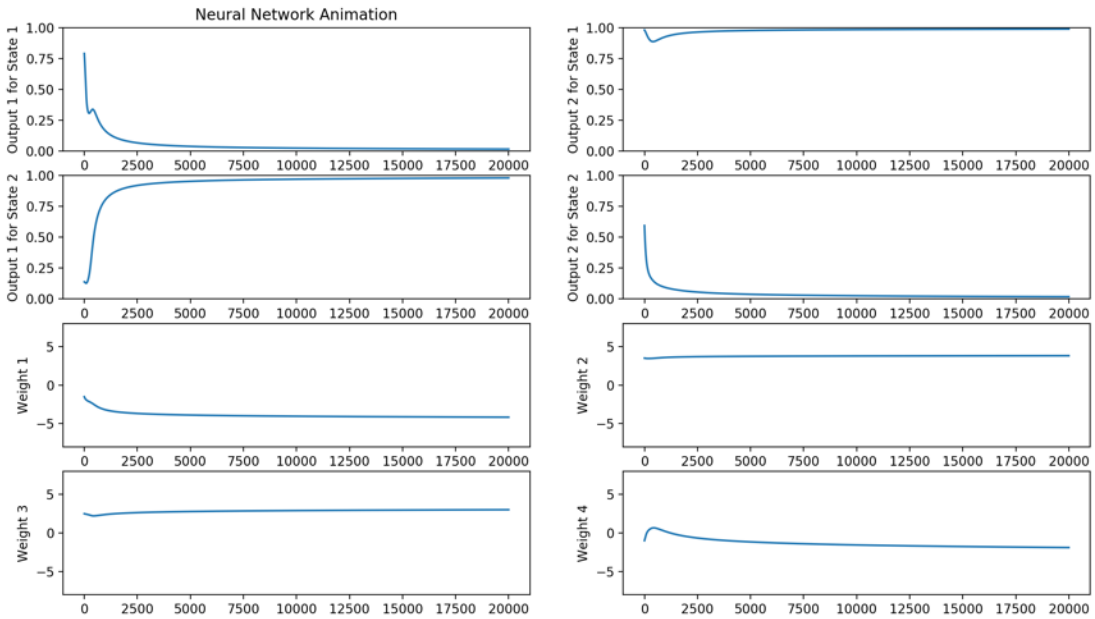

Figure 13. Neural Network Reference System with computation beginning with a first instance of initial Weigh values. This shows evolution of Weight values during each epoch of the Gradient Descent training computation.

A visualization of the evolution of Weight values during training is shown in Figure 14. Here, the Weight values of 1 through 8 are shown plotted as pairs in a 3-dimensional representation with the vertical axis of Neural Network Error. As training proceeds, Weight values are adjusted and Error is reduced. The full behavior over 20,000 Epochs is shown.

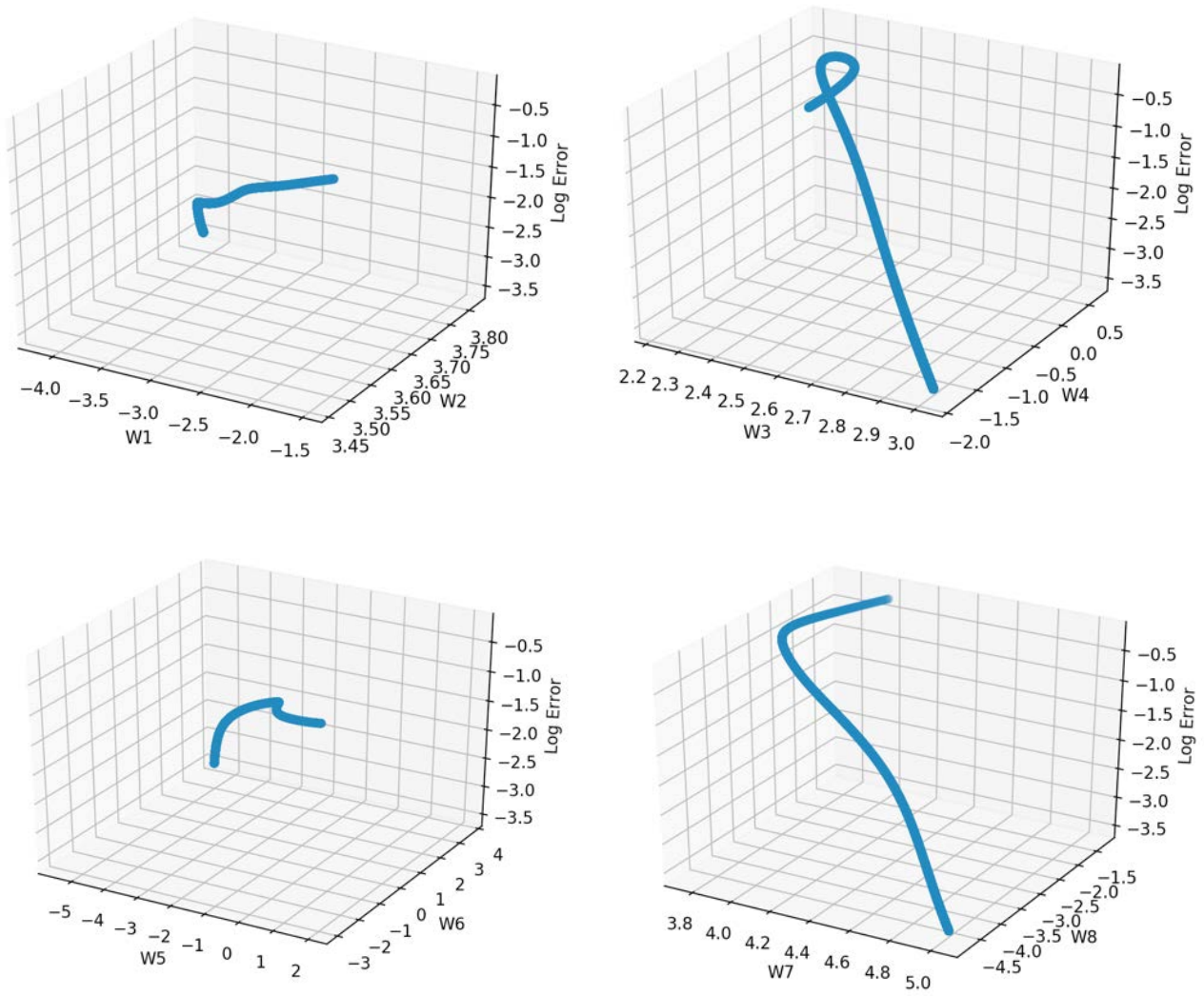

Figure 14. Neural Network Reference System with computation beginning with a first instance of initial Weigh values. This shows evolution of Weight values during each epoch of the Gradient Descent training computation with pairs of Weight values rendered in separate 3 dimensional spaces.

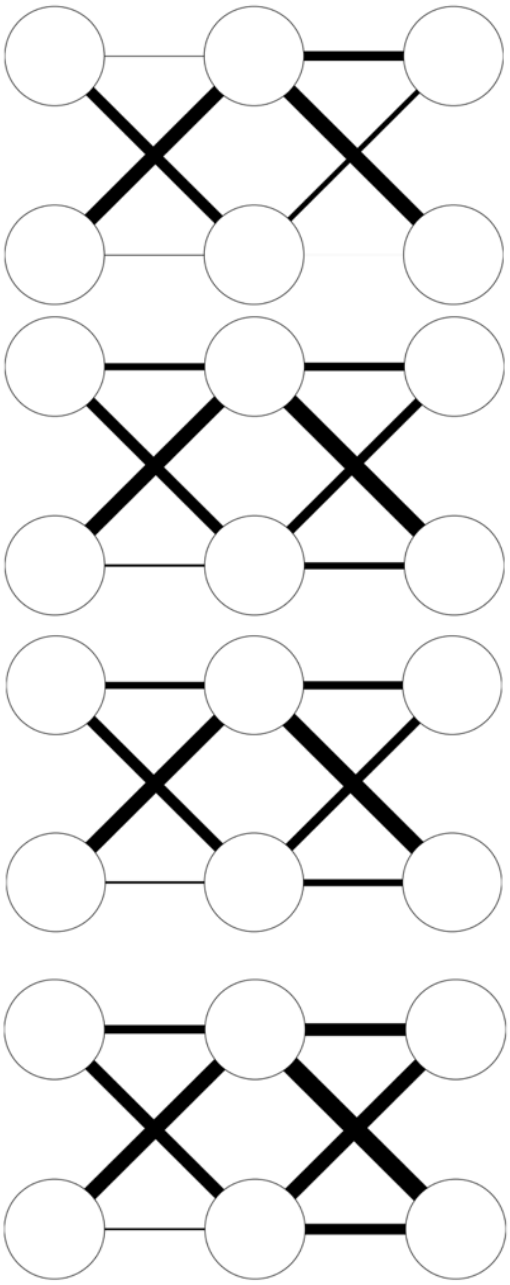

Figure 15. Neural Network Reference System with Weights indicated by width of each synapse line from values of -5 to 5. Epoch 0, 200, 2,000, and 20,000.

Now, the solutions for weight values computed by Gradient Descent may not be unique. Rather, these depend on the initial conditions selected for weight values prior to initiating the Gradient Descent training process. Each selection of initial conditions for weights will generate a unique surface in the n-dimensional space created by the n weights.

Now, to demonstrate that the Neural Network will converge and produce a solution for a differing set of weights satisfying low error, we will initialize the weights with those as above in Table 1, but with one weight value changed. The value of one Weight, W_2 , will be changed from W_2 = -3.5 to W_2 = 3.5 (the polarity of Weight 2 is reversed relative to the first instance).

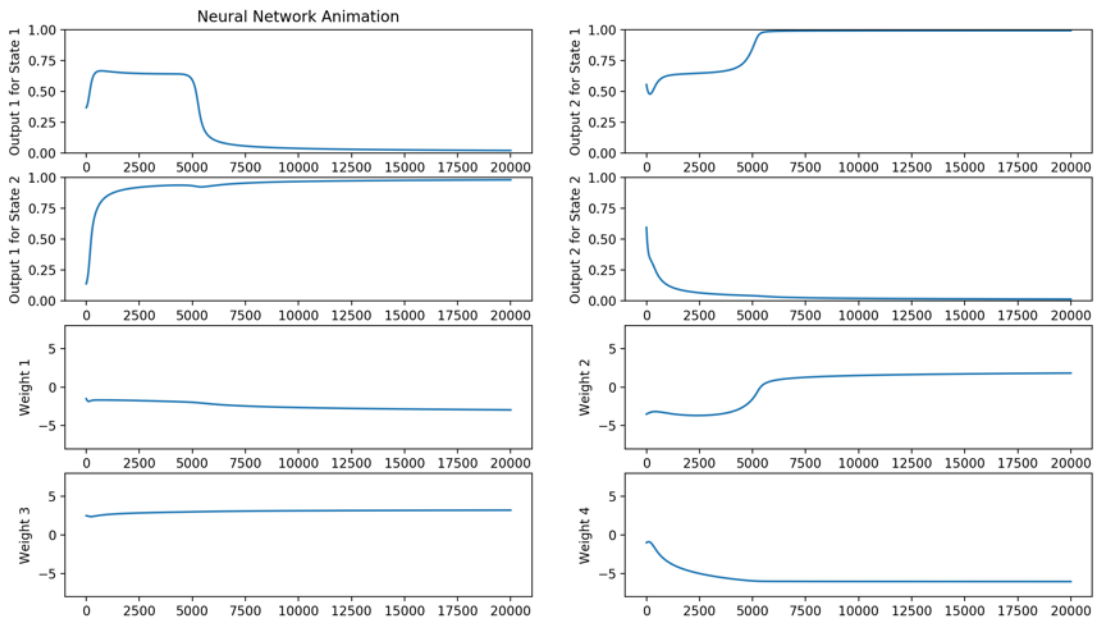

Figure 16. Neural Network Reference System with computation beginning with a second instance of initial Weight values. This shows evolution of Weight values during each epoch of the Gradient Descent training computation.

The results of Neural Network training as observed in Figure 16 are now dramatically different from the previous example with results shown in Figure 13. As an example, the evolution of the value of Output 1 of State 1 begins with a value near 0.3. This value rises, remains constant, and then falls to zero as the Gradient Descent continues. Also, Figure 17 shows the evolution of Weight values during training. The Total Error falls to a value as low as that of the previous example, for different initial Weight values as shown in Figure 14. However, the final Weight values are different. This demonstrates that many paths and selections of Weight values result in small Error and accurate classification. This will be discussed further in Section 11.

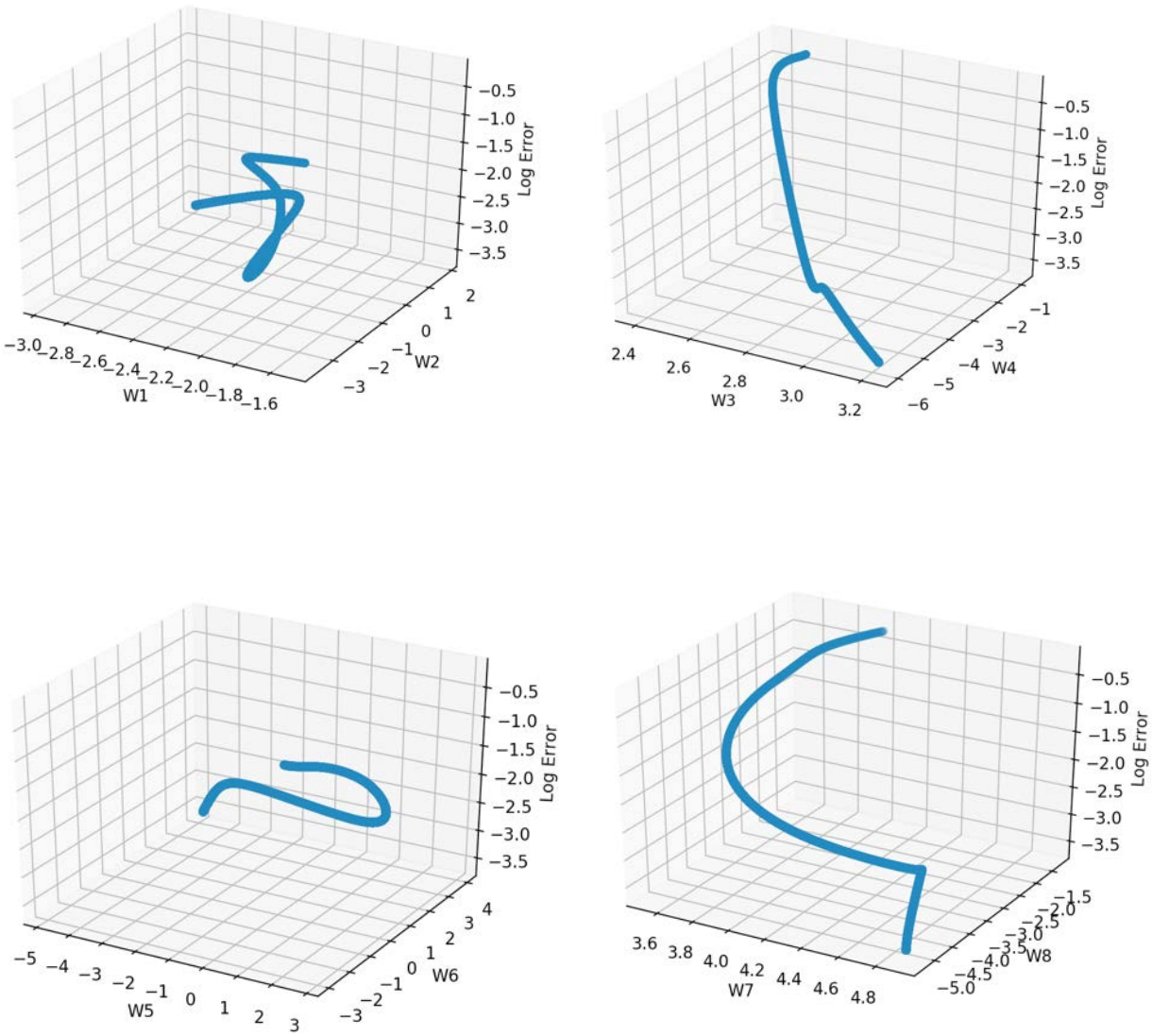

Figure 17. Neural Network Reference System with computation beginning with the second instance of initial Weight values. showing evolution of Weight values during each epoch of the Gradient Descent training computation with pairs of Weight values rendered in separate 3 dimensional spaces.

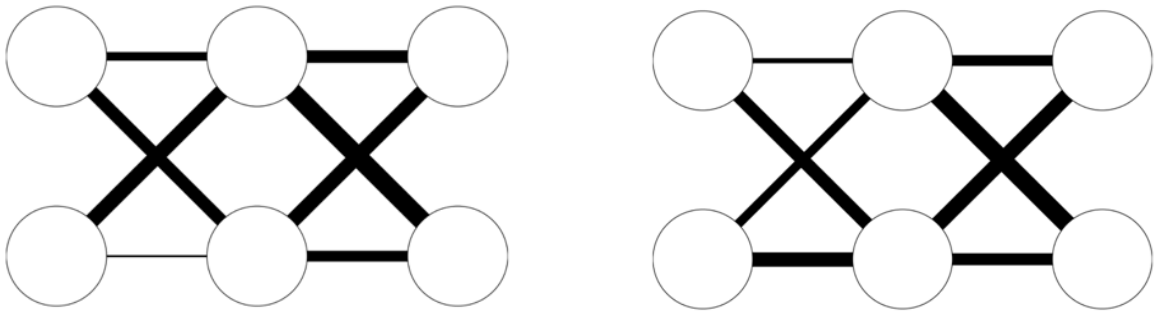

Figure 18. Neural Network Reference System 20,000 Epochs for First Example, and 20,000 Epochs for second example.

10. OBSERVING THE GRADIENT DESCENT

This section describes visualization of the Gradient Descent process. In the previous examples, the Gradient Descent evolution of Weight values occurs simultaneously for all values. Of course, it is not possible to visualize this 8-dimensional space.

Therefore, as a demonstration, Gradient Descent is conducted for an example where 6 of the 8 Weights are maintained constant while two weights are included in Gradient Descent. This permits visualization.

For this demonstration, a set of weight values have been obtained by allowing the Neural Network training to converge after 10,000 epochs to a set of stable values providing low Error.

Then, all Weights have reached optimal values that produce low error.

As a next step, the Weights, W_1 and W_2, are assigned new initial values, differing from the optimal values. Also, the Weights W_3 through W_8 are held constant at their optimal values.

For a range of assigned values, the Total Error is computed for a range of W_1 and W_2 values. This creates a surface in 3-dimensional space shown in Figure 19.

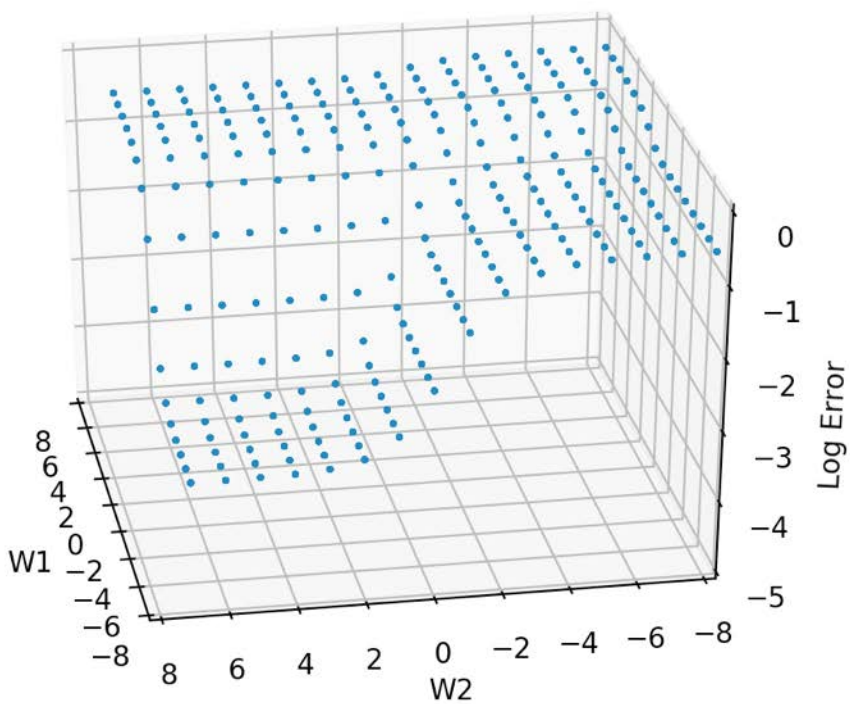

Figure 19. Neural Network Output Error for variation in the Weights, W_1 and W_2. Weights W_3 through of W_8 are held constant at their previously found optimal values. A minimum in this surface appears in the region of $W_1 = 8$ and $W_2 = 6$.

Finally, Gradient Descent training is initiated. The values of W_1 and W_2 are initialized to 4 and -4 respectively. These values are well separated from their optimal values. Thus, a large error results when

forward propagation executes to find output values. However, after 20,000 cycles, the neural network training computes optimal values as shown in Figure 21.

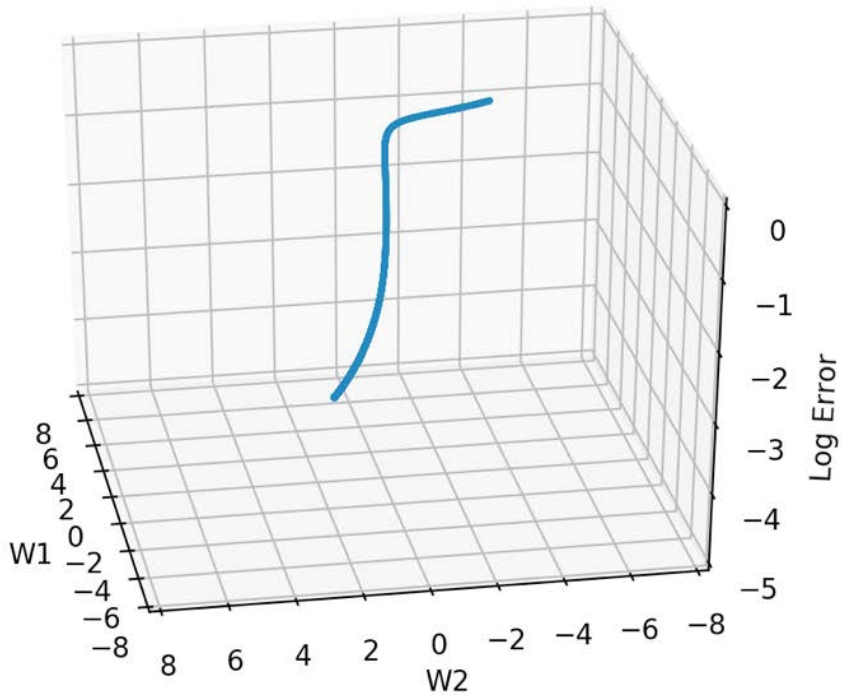

Figure 20. Neural Network Output Error as the Weights of W_1 and W_2 evolve with each Epoch over 20,000 Epochs and result in reduction of Error. Note that this trajectory follows the surface defined in Figure 19.

.

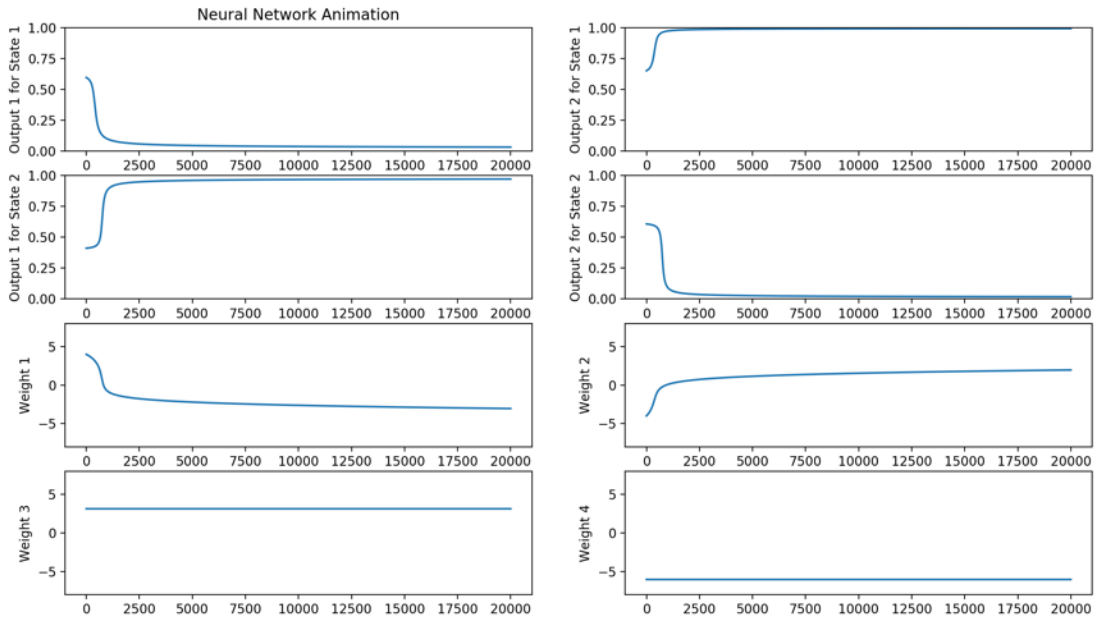

Figure 21. Neural Network Output values for State 1 and State 2 evolve as the Weights of W_1 and W_2 evolve with each Epoch over 20,000 Epochs.

11. LIMITATIONS OF GRADIENT DESCENT

Gradient Descent determination of the unique values of the weights, W_n, may be successful in the limit of networks with few neurons and few layers. The network topology example here, of 2 input neurons, one hidden layer with two hidden layer neurons and two output neurons offers an instance where convergence of Gradient Descent to a minimum corresponding to small Error value is possible.

In general, however, Gradient Descent may find only local minima. As an example, consider the Gaussian function

$$G(x,y) = G_o e^{-\left(\frac{(x-x_o)^2+(y-y_o)^2}{\alpha^2}\right)} \tag{76}$$

In Figure 22, one Gaussian function with $x_o = 0$ and $y_o = 0$ and $G_o = -0.06$ is shown.

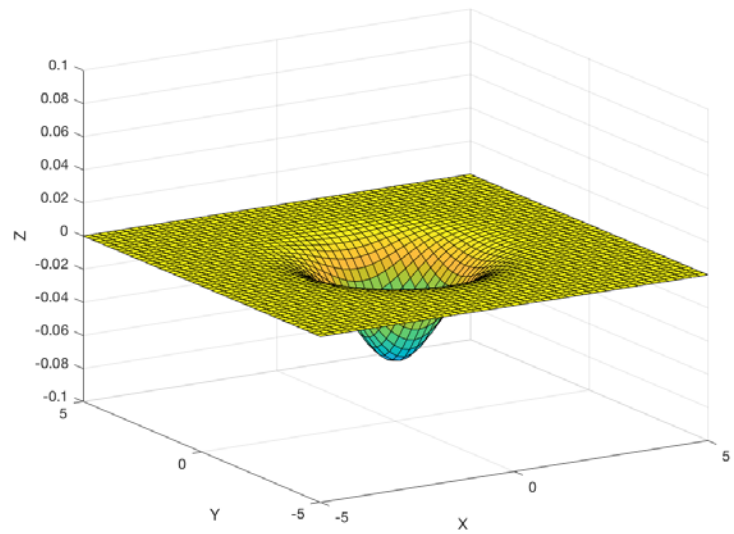

Figure 22. Example surface showing Gaussian function for consideration of Gradient Descent search.

It is clear that initializing a gradient descent search at any location in the region shown will result in a successful search for the global minimum at $x_o = 0$ and $y_o = 0$.

However, consider the more complex surface formed by the sum of first, a Gaussian at $x_{o1} = 0$ and $y_{o1} = 0$ with and $G_{o1} = -0.06$ and a second at $x_{o2} = 1$ and $y_{o2} = 1$ with $G_2 = +0.09$ and a third at $x_{o3} = 2$ and $y_{o3} = 2$ with $G_3 = -0.09$.

$$G(x,y) = G_{o1}e^{-\left(\frac{(x-x_{o1})^2+(y-y_{o1})^2}{\alpha^2}\right)} + G_{o2}e^{-\left(\frac{(x-x_o2)^2+(y-y_{o2})^2}{\alpha^2}\right)} + G_{o3}e^{-\left(\frac{(x-x_{o3})^2+(y-y_{o3})^2}{\alpha^2}\right)} \quad (77)$$

This is shown in Figure 23. It is clear that initializing a gradient descent search at differing locations in the region shown will result in two differing solutions revealing a local minimum at $x = 0$ and $y = 0$ and a global minimum at $x = 2$ and $y = 2$.

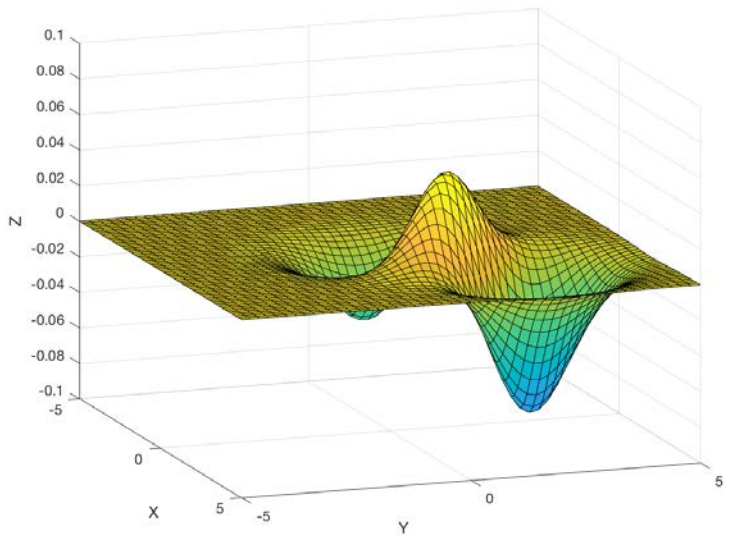

Figure 23. Example surface showing superposition sum of three Gaussian functions for consideration of Gradient Descent search.

12. REFERENCES

[1] http://www.emergentmind.com/the-perceptron and http://www.emergentmind.com/neural-network